

MODULE :- Unit Test Frameworks

EXPERIMENT TITLE :- Page Object Model (POM) Restructure (Assignment 7)

1.Problem Statement :-

To automate a web application login test using **Selenium WebDriver and the Page Object Model (POM)** design pattern. The objective is to separate page elements and their actions into different classes, perform a valid login, and verify that the user is successfully redirected to the inventory page.

2. Tools and Software Required

S.	Tool/Software	Purpose
1	Python	Programming language used for automation
2	VS Code	IDE for developing and running the project
3	Google Chrome	Browser used for testing
4	Selenium WebDriver	Automates browser interactions
5	ChromeDriver	Connects Selenium with Chrome
6	WebDriverWait	Waits for elements to become available
7	Page Object Model	Organizes page elements and actions
8	SauceDemo Website	Application used for login testing
9	Command Prompt / Terminal	Used to run the Python program

3. Concepts Used

- Selenium WebDriver
- **Page Object Model (POM)**
- Page classes
- Base Page
- Locators
- By . ID
- By . NAME
- By . XPATH
- WebDriverWait
- Expected Conditions
- Encapsulation
- Assertions
- Reusable methods
- Test case execution

4.Source Code :-

File 1: base_page.py

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:

    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    def open(self, url):
        self.driver.get(url)

    def find(self, locator):
        return self.wait.until(
            EC.presence_of_element_located(locator)
        )

    def click(self, locator):
        self.wait.until(
            EC.element_to_be_clickable(locator)
        ).click()

    def enter_text(self, locator, text):
        element = self.find(locator)
        element.clear()
        element.send_keys(text)

    def get_current_url(self):
        return self.driver.current_url

    def get_title(self):
```

```
return self.driver.title
```

File 2: login_page.py

```
from selenium.webdriver.common.by import By
from base_page import BasePage
```

```
class LoginPage(BasePage):
```

```
    # -----
    # Locators
    # -----
```

```
    USERNAME = (By.ID, "user-name")
```

```
    PASSWORD = (By.NAME, "password")
```

```
    LOGIN_BUTTON = (
        By.XPATH,
        "//input[@id='login-button']"
    )
```

```
    # -----
    # Page actions
    # -----
```

```
    def enter_username(self, username):
        self.enter_text(
            self.USERNAME,
            username
        )
```

```
    def enter_password(self, password):
        self.enter_text(
            self.PASSWORD,
            password
        )
```

```
    def click_login(self):
        self.click(self.LOGIN_BUTTON)
```

```
    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()
```

File 3: inventory_page.py

```
from selenium.webdriver.common.by import By
from base_page import BasePage
```

```
class InventoryPage(BasePage):
```

```
    PAGE_TITLE = (
        By.CLASS_NAME,
```

```

        "title"
    )

    def get_page_heading(self):
        return self.find(
            self.PAGE_TITLE
        ).text

```

File 4: test_login.py

```
from selenium import webdriver
```

```
from login_page import LoginPage
```

```
from inventory_page import InventoryPage
```

```
def test_valid_login():
```

```
    # Start Chrome
```

```
    driver = webdriver.Chrome()
```

```
    driver.maximize_window()
```

```
    try:
```

```
        # Open SauceDemo
```

```
        driver.get("https://www.saucedemo.com/")
```

```
        # Create Page Objects
```

```
        login_page = LoginPage(driver)
```

```
        inventory_page = InventoryPage(driver)
```

```
        # Perform login
```

```
        login_page.login(
            "standard_user",
            "secret_sauce"
        )
```

```
        # -----
```

```
        # Assertions remain inside the test
```

```
        # -----
```

```
        assert "/inventory.html" in driver.current_url
```

```
        print("PASS: Login successful")
```

```
        print("PASS: URL contains /inventory.html")
```

```
        # Get inventory page heading
```

```
        print(
            "Page heading:",
            inventory_page.get_page_heading()
        )
```

```
    finally:
```

```
        # Close browser
```

```
        driver.quit()
```

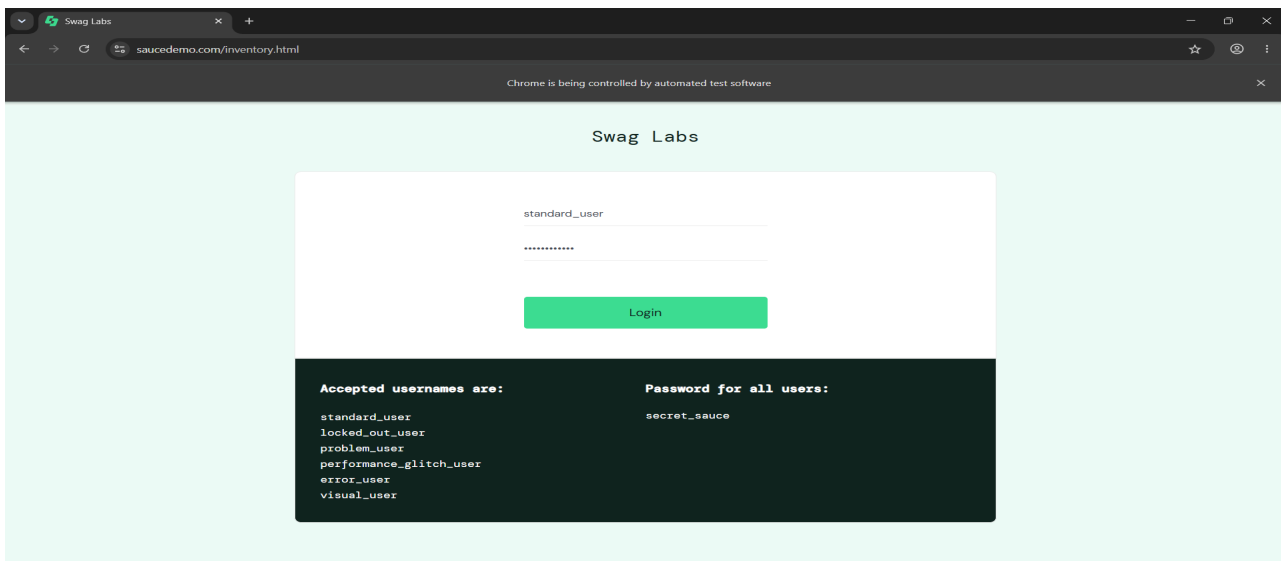
```
# =====
# RUN THE TEST WHEN THIS FILE IS EXECUTED DIRECTLY
# =====

if __name__ == "__main__":
    test_valid_login()
```

5.Implementation Steps :-

- 1.Install Python and Selenium WebDriver.
- 2.Create a separate project folder for Assignment 7.
- 3.Create `base_page.py` to contain common Selenium operations such as finding elements, clicking and entering text.
- 4.Create `login_page.py` to store the login page locators and login-related methods.
- 5.Create `inventory_page.py` to store elements and methods related to the inventory page.
- 6.Create `test_login.py` to contain the actual login test case.
- 7.Open the SauceDemo website using Selenium.
- 8.Create an object of the `LoginPage` class.
- 9.Enter the username and password using the methods defined in the Login Page.
- 10.Click the Login button.
- 11.Verify that the URL contains `/inventory.html`.
- 12.Create an object of the `InventoryPage` class.
- 13.Retrieve and display the **Products** page heading.
- 14.Close the browser after execution.
- 15.Run the test using the Python command in the terminal.

6.Output :-



```
PS C:\Users\catch\OneDrive\Desktop\wipro_assignments\assignment_7> python test_login.py
PASS: Login successful
PASS: URL contains /inventory.html
Page heading: Products
PS C:\Users\catch\OneDrive\Desktop\wipro_assignments\assignment_7>
```

7.Result :-

The login test was successfully executed using the **Page Object Model**.

- Login was successful with valid credentials.
- The URL contained `/inventory.html`.
- The inventory page heading "**Products**" was successfully retrieved.
- The browser was closed successfully after execution.
-

8.Observation :-

The Page Object Model separates **page elements, page actions, and test logic** into different classes. This makes the automation code more organized, reusable, and easier to maintain. Common operations were placed in the Base Page and reused by other page classes.

9.Conclusion :-

The **Page Object Model (POM)** was successfully implemented with Selenium WebDriver and Python. The experiment demonstrated how separating page-related elements and actions from test cases improves the structure and reusability of Selenium automation scripts.